Pune Vidyarthi Griha's
COLLEGE OF ENGINEERING, TECHNOLOGY AND MANAGEMENT, PUNE-09

Name : **Class : B.E.** **Roll No :**

PRACTICAL NO. 02

Uber Fare Prediction using Linear, Ridge and Lasso Regression

Aim

To pre-process the Uber Fares dataset (clean the data and identify fare outliers), inspect the correlations between features, and fit Linear, Ridge and Lasso regression models to predict the fare, comparing them using R^2, RMSE and MAE.

Code and Output

In [1]:
```
import os, numpy as np, pandas as pd, matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression, Ridge, Lasso
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
```

In [2]:
```
DATA_PATH = "uber.csv"  # rename your downloaded CSV to this
if not os.path.exists(DATA_PATH):
    raise FileNotFoundError("Place the Uber Fares CSV beside this notebook "
                            "as uber.csv")
df = pd.read_csv(DATA_PATH)
df.columns = [c.strip().lower() for c in df.columns]
display(df.head())
print("Original shape:", df.shape)
```

```
   unnamed: 0                            key  fare_amount  \
0    24238194    2015-05-07 19:52:06.0000003          7.5
1    27835199    2009-07-17 20:04:56.0000002          7.7
2    44984355   2009-08-24 21:45:00.00000061         12.9
3    25894730    2009-06-26 08:22:21.0000001          5.3
4    17610152  2014-08-28 17:47:00.000000188         16.0

           pickup_datetime  pickup_longitude  ...
0  2015-05-07 19:52:06 UTC        -73.999817  ...
1  2009-07-17 20:04:56 UTC        -73.994355  ...
2  2009-08-24 21:45:00 UTC        -74.005043  ...
3  2009-06-26 08:22:21 UTC        -73.976124  ...
4  2014-08-28 17:47:00 UTC        -73.925023  ...

Original shape: (200000, 9)
```

In [3]:
```
target = "fare_amount"
if target not in df.columns:
    raise ValueError(f"Expected target column '{target}'. "
```

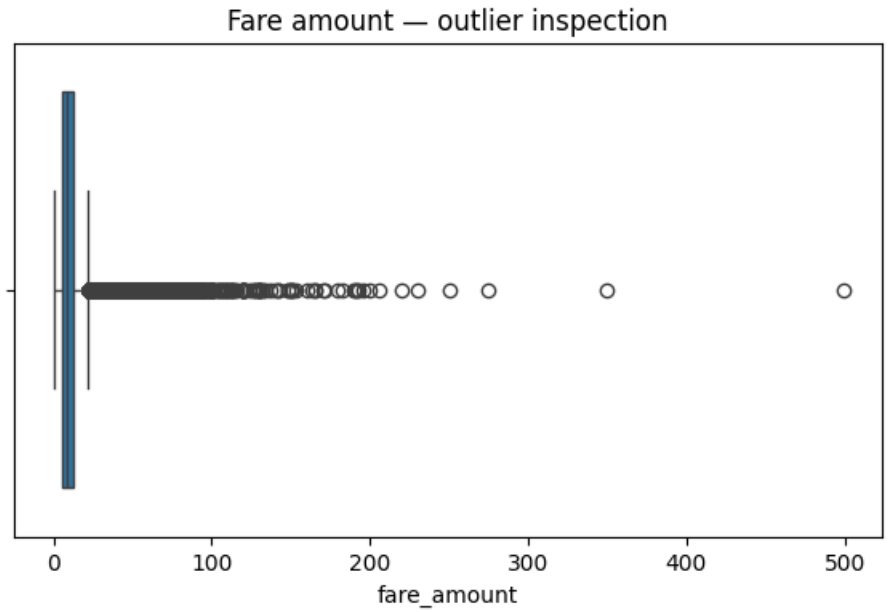
```
                     f"Found: {list(df.columns)}")
df = df.replace([np.inf, -np.inf], np.nan).dropna(subset=[target])
df = df[df[target] >= 0]
for col, lo, hi in [("pickup_longitude", -180, 180),
                    ("dropoff_longitude", -180, 180),
                    ("pickup_latitude", -90, 90),
                    ("dropoff_latitude", -90, 90)]:
    if col in df.columns: df = df[df[col].between(lo, hi)]
if "passenger_count" in df.columns:
    df = df[df["passenger_count"].between(1, 6)]
if "pickup_datetime" in df.columns:
    dt = pd.to_datetime(df["pickup_datetime"], errors="coerce", utc=True)
    df["pickup_hour"] = dt.dt.hour
    df["pickup_day"] = dt.dt.day
    df["pickup_month"] = dt.dt.month
    df["pickup_weekday"] = dt.dt.dayofweek
    df = df.drop(columns=["pickup_datetime"])
df = df.select_dtypes(include="number").dropna(subset=[target])
q1, q3 = df[target].quantile([.25, .75]); iqr = q3 - q1
outlier_mask = (df[target] < q1 - 1.5 * iqr) | (df[target] > q3 + 1.5 * iqr)
print("Rows after basic cleaning:", len(df))
print("Fare outliers by IQR rule:", int(outlier_mask.sum()))
display(df.describe().T)
```

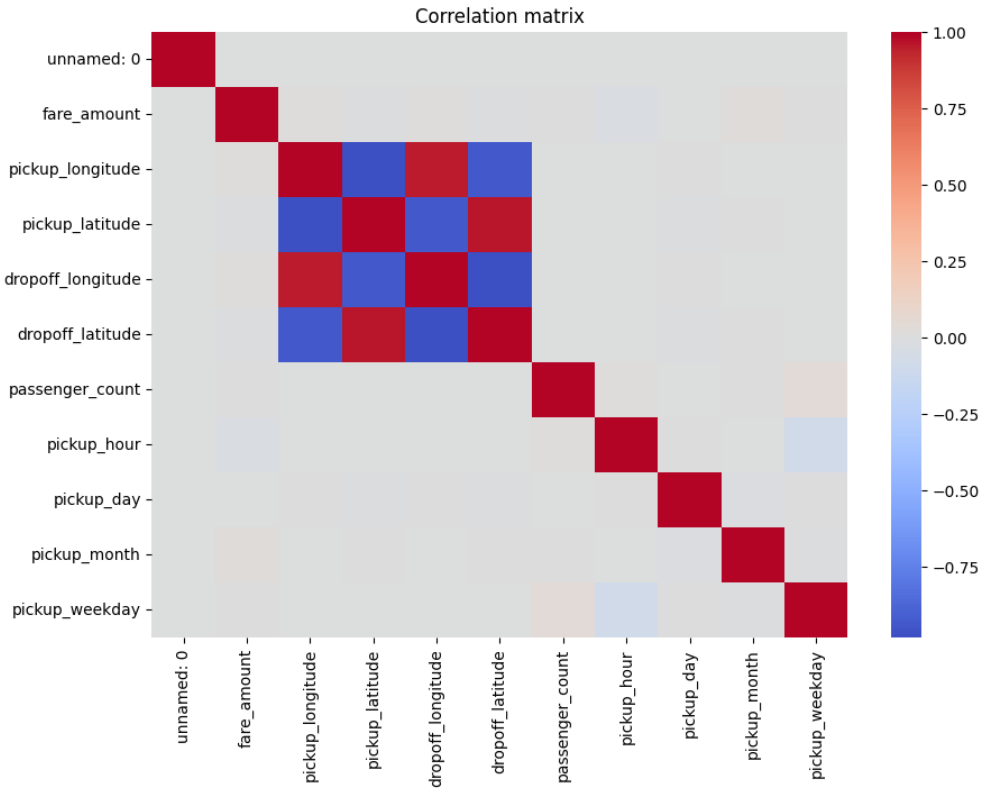
```
Rows after basic cleaning: 199261
Fare outliers by IQR rule: 17120
```

	count	mean	std	min	...
unnamed: 0	199261.0	2.771598e+07	1.601407e+07	1.000000	...
fare_amount	199261.0	1.136909e+01	9.906025e+00	0.000000	...
pickup_longitude	199261.0	-7.250384e+01	1.044338e+01	-93.824668	...
pickup_latitude	199261.0	3.991899e+01	6.128334e+00	-74.015515	...
dropoff_longitude	199261.0	-7.251370e+01	1.040545e+01	-75.458979	...
dropoff_latitude	199261.0	3.992310e+01	6.115539e+00	-74.015750	...
passenger_count	199261.0	1.689453e+00	1.305393e+00	1.000000	...
pickup_hour	199261.0	1.349267e+01	6.516173e+00	0.000000	...
pickup_day	199261.0	1.570564e+01	8.686897e+00	1.000000	...
pickup_month	199261.0	6.283367e+00	3.438473e+00	1.000000	...
pickup_weekday	199261.0	3.049382e+00	1.946785e+00	0.000000	...

In [4]:

```
plt.figure(figsize=(7,4))
sns.boxplot(x=df[target])
plt.title("Fare amount - outlier inspection"); plt.show()
corr = df.corr(numeric_only=True)
plt.figure(figsize=(10,7))
sns.heatmap(corr, cmap="coolwarm", center=0)
plt.title("Correlation matrix"); plt.show()
print("Correlation with fare:")
display(corr[target].sort_values(ascending=False).to_frame("correlation"))
```





Correlation with fare:

	correlation
fare_amount	1.000000
pickup_month	0.023509
dropoff_longitude	0.011899
pickup_longitude	0.011558
passenger_count	0.010173
pickup_weekday	0.007213
pickup_day	0.001439
unnamed: 0	0.000613
pickup_latitude	-0.011485
dropoff_latitude	-0.012282
pickup_hour	-0.021460

In [5]:

```
# Use a sample for faster practical execution on very large CSVs.
if len(df) > 100_000:
    df = df.sample(100_000, random_state=42)
X, y = df.drop(columns=[target]), df[target]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=.2, random_state=42)
models = {"Linear Regression": LinearRegression(), "Ridge": Ridge(alpha=1.0),
          "Lasso": Lasso(alpha=.01, max_iter=10000)}
rows = []
for name, estimator in models.items():
    pipe = Pipeline([("imputer", SimpleImputer(strategy="median")),
                     ("scaler", StandardScaler()), ("model", estimator)])
    pipe.fit(X_train, y_train)
    pred = pipe.predict(X_test)
    rows.append({"Model":name, "R2":r2_score(y_test,pred),
                 "RMSE":np.sqrt(mean_squared_error(y_test,pred)),
                 "MAE":mean_absolute_error(y_test,pred)})
results = pd.DataFrame(rows).sort_values("R2", ascending=False)
display(results.round(4))
```

	Model	R2	RMSE	MAE
2	Lasso	0.0013	10.0735	6.0905
1	Ridge	0.0013	10.0738	6.0910
0	Linear Regression	0.0013	10.0738	6.0910